

SETUP GUIDE

rahhh

Twitch Bot Control Panel

Everything you need to connect your Twitch accounts, start the bot, and turn on whichever optional features you want: Spotify song requests, OBS overlays, the Wordle chat game, Daily Check-In, custom commands, and more.

 Song Requests

 Wordle

 Daily Check-In

 Coin Toss

 Clips

 Custom Commands

Start with Section 1, even if you only care about one feature: the Twitch connection is required for everything else.

CONTENTS

What's in this guide

01 Before You Start

What you'll need on hand before opening the app

Required

02 Connecting Twitch

Streamer, Bot, and Mod accounts

Required

03 Starting the Bot

A separate step from connecting accounts

Required

04 Spotify & Song Requests

!sr, channel points, playlist auto-add, Now Playing overlay

Optional

05 OBS Overlays

SFX, Now Playing, and Wordle browser sources

Optional

06 Wordle Game

Chat-driven Wordle with a leaderboard

Optional

07 Daily Check-In

A channel-points redeem that awards points, sent from your own account

Optional

08 Custom Commands

Build your own !commands, no code required

Optional

09 Channel Point Redeems

Create and manage rewards directly from the app

Optional

10 Coin Toss

Zero-config chat minigame, nothing to set up

Optional

11 Troubleshooting

The mistakes almost everyone makes once

Handy

SECTION 01



Before You Start

A five-minute checklist before you open the app for the first time.



You'll always need: a free Twitch account to register a Developer app with (this can be your streaming account or a separate one; you'll paste its Client ID/Secret into the app, nothing gets uploaded anywhere else).



Only if you want Song Requests: a free Spotify Developer app, and Spotify Premium on the account you connect (the Spotify Web API's playback controls, like queueing and skipping, require Premium).

How this app is organized

The sidebar has one page per feature: **Dashboard**, **Wordle**, **Lists**, **Clips**, **Daily Check In**, **Song Requests**, **Commands**, **Redeems**, and **Settings**. Everything under Settings is required. Everything else is optional and only does anything once you turn it on.

The general shape of setup

1. Connect your Twitch accounts in **Settings** (Section 2).
2. Press **Start Bot** on the **Dashboard** (Section 3); this is a separate step people often miss.
3. Turn on whichever optional features you actually want. Everything else can stay off forever with zero downside.



Connecting Twitch

Everything lives on the **Settings** page. This is the only section every single user has to do, regardless of which optional features they want.

Why three separate Twitch accounts?

The app connects to **three** Twitch logins, each doing a distinct job:

Streamer Account

Sets the **Target Channel** the bot operates in, and sends official shoutouts.

Bot Account

The login the bot actually chats as. Can be a dedicated bot account, or the same login as your Streamer account if you don't have one.

Mod Account

Used only for creating/downloading clips. Sets **Moderator Login**.

All three can be the exact same Twitch login if you don't have separate accounts. The app doesn't require them to differ, it just supports it if you do.

Step-by-step

- 1 Open the **Settings** page and find the **Twitch App** section.
- 2 Click **Open Twitch Developer Console**, sign in, and register a new application.
- 3 In your new Twitch app's settings, add this exact URL to **OAuth Redirect URLs**:

`http://localhost:17846/callback`

The app also shows this value next to a **Copy** button. Use that instead of retyping it, since one wrong character here is the #1 cause of "Connect" not working.

- 4 Copy your new app's **Client ID** and **Client Secret** back into the matching fields in Settings.
- 5 Click **Connect with Twitch** under **Streamer Account**, approve access in the browser window that opens, and confirm the login. Repeat for **Bot Account** and **Mod Account**.



Redirect URI must match exactly. Twitch's console is picky about trailing slashes and http vs https. If a Connect button seems to hang or silently fail, this is almost always why. Go double-check it against the Copy button's value.



Each Connect button forces Twitch's account picker to show up (`force_verify`), even if you're already logged into Twitch in your browser, so you can safely connect three different logins one after another without having to log out in between.

Access/Refresh tokens for each account are shown (password-hidden, with a Show toggle) and refresh themselves automatically from then on. You only go through this flow again if you disconnect or a token gets



Starting the Bot

This trips up almost everyone once: connecting your Twitch accounts does **not** start the bot.

Go to the Dashboard

Once all three Twitch accounts are connected, head to **Dashboard** and click the big  button. This launches the actual bot process (chat listening, EventSub subscriptions, everything) as a separate step from just having your accounts connected.



Relaunching the whole app is not the same as restarting the bot. If the app itself restarts (you closed and reopened it, or it auto-updated), the bot does *not* come back on its own. You have to press Start Bot again.

When do I need to restart the bot?

As a rule of thumb:

- **Always** after changing anything on the Settings page. The  button there handles this for you automatically.
- **Usually not** for most other pages (Song Requests, Wordle, Daily Check-In, Commands). Those settings are picked up live or the next time they're relevant. If something you just changed doesn't seem to take effect, a quick Stop → Start on the Dashboard always resolves it.



Spotify & Song Requests

Lets viewers request songs by chat command or channel points, with an optional live Now Playing overlay for OBS.

Connect Spotify

On the **Song Requests** page, find the **Spotify Connection** section:

- 1 Click **Open Spotify Developer Dashboard** and create an app.
- 2 Add this exact redirect URI to your Spotify app's settings:
`http://127.0.0.1:17845/callback`
- 3 Paste your Spotify app's Client ID and Client Secret into the matching fields, then click **Connect Spotify**.

⚠ Spotify's redirect URI uses `127.0.0.1`, not `localhost`: the opposite convention from the Twitch one in Section 2. Spotify's own console rejects plain `localhost`, so this isn't a typo.

🎧 The connected Spotify account needs Premium: queueing, skipping, and playlist edits all go through Spotify's playback-control API, which free accounts can't use.

Turn on requests

Under **Request Behavior**, enable either or both:

- **Chat command**: defaults to `!sr`, editable.
- **Channel points redeem**: pick an existing Twitch reward from the dropdown (create it on Twitch itself, or on this app's **Redeems** page, first).

From there you can set a per-user cooldown, a max song length, how many seconds before a track ends the next request gets pushed into Spotify's queue, how many upcoming songs `!queue` shows by default, who's allowed to use the chat command (by role/sub tier), and a max-pending-requests limit per role.

Requests understand more than plain text

Viewers can paste a Spotify track link directly, or type `song title by artist` for the most reliable match. Both skip Spotify's fuzzy text search, which is what a plain `!sr` query falls back to.

Auto-add to a playlist

The **Playlist** dropdown lists every playlist on the connected account automatically, plus a synthetic **♥ Liked Songs** entry. Pick one and mods/broadcaster can run `!add` to add whatever's currently playing to it.

Chat commands this unlocks

COMMAND	WHO	WHAT IT DOES
---------	-----	--------------

<code>!sr <song></code>	Configurable	Requests a song
<code>!remove</code>	Requester / Mod	Cancels a still-pending request
<code>!song</code>	Everyone	Shows what's currently playing
<code>!queue [N]</code>	Everyone	Shows upcoming songs
<code>!next</code>	Everyone	Shows just the next song
<code>!add</code>	Mod/Broadcaster	Adds the current song to your chosen playlist
<code>!skip</code>	Mod/Broadcaster	Skips the current song, promoting the next request first

The **OBS Now Playing Overlay** subsection has its Browser Source URL and a **Card alignment** (Left/Center/Right) setting; see Section 5 for adding it to OBS.



OBS Overlays

Three independent browser-source overlays, each opt-in: nothing happens unless you actually add its URL to OBS.

OVERLAY	WHERE THE URL IS	WHAT IT SHOWS
SFX	Settings → SFX Browser Source URL	Plays every sound effect (custom commands, redeems)
Now Playing	Song Requests → OBS Now Playing Overlay	Album art, title, artist, progress bar
Wordle	Wordle → Browser Source URL	The live Wordle board

Adding one to OBS

- 1 Click the **Copy** button next to the overlay's URL.
- 2 In OBS: **Sources** → + → **Browser**, paste the URL.
- 3 Set a Width/Height that comfortably fits the content, then position it in your scene.

All three overlays push live updates: a freshly added or reconnected browser source shows the current state immediately instead of sitting blank until something changes.



If text looks soft/blurry: set the Browser Source's Width/Height in OBS to *double* what you actually want on screen, then scale the source back down to 50% in your scene (drag a corner, or set Transform → Scale). This renders at higher pixel density and lets OBS downscale it, which is the standard fix for fine text looking fuzzy in a Browser Source.

For SFX specifically: drop `.mp3` / `.wav` / `.ogg` files into the app's `sfx` folder, or use the file picker when setting up a Custom Command or Redeem that plays a sound. Either way, they show up automatically, no need to touch the overlay itself.

Wordle Game

A chat-driven Wordle round your viewers guess together, with its own leaderboard and OBS board.

Settings

On the **Wordle** page: a start/stop chat command (defaults to `!sw`, mod/broadcaster-only), round duration in seconds (default 105), and the delay before the next round auto-starts after a win or timeout (default 10s).

Commands

COMMAND	WHO	WHAT IT DOES
<code>!sw</code>	Mod/Broadcaster	Turns the whole game on/off
<code>!pause</code>	Mod/Broadcaster	Pauses the current round's timer
<code>!play</code> / <code>!resume</code>	Mod/Broadcaster	Resumes a paused round
<code>!reset</code> / <code>!restart</code>	Mod/Broadcaster	Force-starts a new round
<code>!topwins</code>	Everyone	Shows the top 5 leaderboard
<code>!help</code>	Everyone	Explains how to play

Every message above is individually editable from the **Commands** page.

Coming from the old standalone Wordle app?

The **Leaderboard** tab has an `Import Legacy Leaderboard` button that pulls win counts from the old standalone game's save file and merges them in. It only ever raises a count, never lowers one, so it's safe to run more than once.



Daily Check-In

A single channel-points redeem that awards a viewer points and replies to them, sent from **your own** Twitch account, not the bot's.

Setup

- 1 Create the reward on Twitch first (or on this app's **Redeems** page).
- 2 On the **Daily Check In** page, enable it and pick that reward from the dropdown.
- 3 Set how many points a check-in is worth, and the two-line chat message that gets sent. Placeholders `{user}` and `{value}` are available in both lines.



The confirmation is sent as your **streamer account**, not the bot, so it reads like you personally welcoming them. This needs your Streamer connection from Section 2 to have gone through cleanly.

The **Check-In Points** tab is a searchable, editable list of everyone's running total, the same numbers `!ssnacks` and `!topsnacks` read from in chat. A generic Redeem (Section 9) can also be set to award these points independently, if you want more than one way to earn them.



Custom Commands

Build your own chat commands with no code, and edit the wording of the bot's built-in ones.

Built-in commands

The **Commands** page lists everything the bot ships with. Anything with an **Edit messages** button lets you rewrite its exact chat replies. Most of the built-in commands support this.

Making your own

Click **+ New Command** and fill in:

- **Trigger:** e.g. `!lurk`
- **Type:** *Reply* (one or more random variants, with placeholders like `{user}`, `{args}`, `{random:1-100}`, `{choice:a|b|c}`) or *Counter* (a per-command number that increments/decrements/displays, with a `{value}` placeholder)
- **Sound effect:** optional, pick a file and a volume
- **Who can use it:** Everyone, any sub tier, Moderator, or Broadcaster (broadcaster always passes regardless)
- **Cooldown:** none, global, or per-user, in seconds



Channel Point Redeems

Create and manage Twitch channel-point rewards directly from the app, and wire them up to do something automatically.

Click **+ New Redeem** to set a title, cost, viewer-facing prompt, whether it needs text input, its color, and whether it's visible/instantly-fulfilled on Twitch. Then pick a **Bot Behavior**:

ACTION	WHAT HAPPENS
Message	Bot replies in chat
Counter	Adjusts a named counter, same ones Custom Commands can read
Snack Check-in	Awards Daily Check-In points, same store as Section 7
Sound Effect	Plays a sound on the SFX overlay

Rewards that already existed on Twitch before you added them here can still have their bot behavior configured, even though their title/cost/prompt stay editable only on Twitch itself.



Turning on **Skip Reward Queue** means Twitch auto-fulfills the redemption before the bot ever sees it, so anything this app would normally enforce (like a max-length check on a song request redeem) can't actually block it.

SECTION 10 · OPTIONAL



Coin Toss

A fully automatic chat minigame: there's genuinely nothing to configure.

Every so often while you're live, a random viewer gets challenged in chat and has 30 seconds to reply `!coin heads` or `!coin tails` for a chance at points. It's entirely passive. The only control you have is:

COMMAND	WHO	WHAT IT DOES
<code>!p</code>	Mod/Broadcaster	Pauses or resumes the whole game



Points are awarded through an external `!givepoints`-style command. This assumes you already have a separate points/currency bot running (e.g. StreamElements) that owns that command.

SECTION 11



Troubleshooting

The handful of mistakes that account for almost every "it's not working" report.

SYMPTOM	LIKELY CAUSE
A Connect button hangs, or the browser tab shows an error	Redirect URI mismatch. Twitch uses <code>localhost:17846</code> , Spotify uses <code>127.0.0.1:17845</code> . Don't mix them up, and copy rather than retype.
Accounts are connected but nothing happens in chat	The bot itself was never started. Go press Start Bot on the Dashboard (Section 3); this is by far the most common one.
Everything worked yesterday, nothing works today	If the whole app restarted (update, reboot, closed and reopened), the bot needs to be started again manually. It doesn't resume on its own.
Spotify features suddenly return a permission/scope error	An app update may have added a new Spotify permission. Refreshing your existing connection can't grant a new scope. You need to fully reconnect Spotify (Section 4) again.
A channel-points reward isn't showing up in a dropdown	Make sure it actually exists on Twitch (or was created on the Redeems page) before trying to select it, then reopen the page so it re-fetches the list.
Something changed in Settings but doesn't seem to apply	Stop and Start the bot again from the Dashboard. Settings-page changes need a restart, and the app's own Save button there does this for you automatically.

Quick reference: ports & overlay URLs

WHAT	VALUE
Twitch OAuth callback	<code>http://localhost:17846/callback</code>
Spotify OAuth callback	<code>http://127.0.0.1:17845/callback</code>
SFX overlay	Settings page
Now Playing overlay	Song Requests page
Wordle overlay	Wordle page



Still stuck? Every field in this app has a short hint underneath it. When in doubt, that's the fastest source of truth, since it always matches whatever version you're actually running.